

Special Participation C: Refactoring Q5 on HW4 – REPORT

By Ken Zheng

Introduction

This report documents the refactoring of the `HandDesignFilters_Sol.ipynb` colab notebook into refactored github repo: `hand_design_filters`: https://github.com/kenzhengjk/hand_design_filters. The original notebook combined data loading, visualization, model logic, and execution flow into a single linear document. While suitable for quick demos, the monolithic structure violates several software engineering principles, making the code difficult to maintain and test.

My refactored codebase adopts a modular "src-layout" structure, adheres to [PEP 8](#) style guidelines, utilizes [PEP 484](#) type hinting, and implements unit testing. These changes align with industry standards for machine learning and general software engineering practices, such as those outlined in [NumPy's Style Guide](#), [Google's Python Style Guide](#), and standard Python packaging guides.

New Project Structure

The project has been reorganized from a single `.ipynb` file into a standard Python package structure for better scalability and user (student) experience.

Directory Tree (`.py` files only):

```
None
hand_design_filters/
├── src/
│   ├── __init__.py
│   ├── filters.py          # Core logic
│   └── utils.py           # Helper functions
├── tests/
│   ├── __init__.py
│   └── test_filters.py     # Unit tests (new addition!)
└── run.py                 # Entry point script
```

Detailed Refactoring Changes

1. Core Logic: `src/filters.py`

The primary image processing algorithms: blur and edge detection, were extracted from the notebook cells and encapsulated into pure functions within `filters.py`. I also added a third task asking for a sharpening filter. It seemed a reasonable addition since we had a blur filter from task 1 that we could incorporate into our sharpening filter in an interesting way!

- **Encapsulation:** Functions explicitly accept all necessary data as arguments (e.g., `get_sharpen_kernel(blur_kernel, sharpness_factor)`).
- **Type Hinting:** In accordance with **PEP 484**, all function signatures now include type hints (e.g., `image: np.ndarray`) to improve readability and enable “typed” analysis.
- **Documentation:** Docstrings were rewritten to follow the **NumPy Style Guide** and **Google Python Style Guide**, e.g., describing `Args` and `Returns` clearly.

Example Change: *Original Notebook:*

```
Python
def edge_detecting(image):

#####
#####
    # TODO: Implement the Laplacian derivative filter.
#

#####
#####
```

Refactored (`filters.py`):

```
Python
def get_edge_detection_kernel() -> np.ndarray:
    """
    Constructs a 3x3 edge detection kernel (Laplacian).

    Returns:
        A 3x3 numpy array.
    """
    ...
```

```
def apply_convolution(image: np.ndarray, kernel: np.ndarray) ->
np.ndarray:
    """
    Applies a convolution kernel to an image.

    Args:
        image: 2D numpy array representing the grayscale image.
        kernel: 2D numpy array representing the filter kernel.

    Returns:
        The convolved image.
    """
    ...
```

2. Utilities: `src/utils.py`

Auxiliary code handling I/O and visualization was moved to `utils.py`.

- **Separation of Concerns:** The original notebook mixed plotting code (`plt.imshow`) with algorithmic logic. By moving `load_image_from_url`, `rgb2gray`, and `show_multiple_images`, etc. to a separate module, the core logic in `filters.py` now is "pure" and testable without any GUI dependencies!
- **Reproducibility:** A `set_seed` function was added to centralize random seed control (Torch, NumPy, and Random), adhering to ML reproducibility best practices.

3. Execution: `run.py`

The linear execution flow of the colab notebook was converted into a script: `run.py`.

- **Orchestration:** This script imports functions from `src` and orchestrates the pipeline (i.e., Load Data → Task 1 → Task 2 → Task 3).
- **Error Handling:** `try-except` blocks were added around student tasks to gracefully handle `NotImplementedError`, (theoretically) improving the student experience compared to colab's crash-on-failure behavior.

4. Testing: `tests/test_filters.py`

A critical addition to the software engineering process is the `tests` directory.

- **Unit Testing:** The original notebook relied on eye-balling (e.g., “Does the image look blurry?”). The refactored code introduces `unittest` cases that verify mathematical properties of the filters (e.g., “Does the blur kernel weights sum to 1.0?”).
- **Automated Verification:** Though a bit extra for the purposes of a homework assignment, this creates a good base for continuous integration workflows, a popular practice in SWE that is difficult to achieve with `.ipynb` files.

Summary

The refactoring process transformed a scratch-book-styled colab notebook into a robust assignment following good software engineering practices. By applying **PEP 8** formatting, enforcing **PEP 484** typing, adopting a **modular file structure**, among other changes/additions, the code is now more readable, maintainable, and testable: all definite pros for future students tackling the problem. And the addition of unit tests ensures that the teaching value of implementing basic image processing filters is preserved while verifying correctness mathematically rather than just visually!

LLM Usage

Most of the refactoring was done by hand, given the original colab notebook was not too convoluted (compared to some other coding questions/demos involving training). I prompted *Claude Sonnet 4.5* to create the `README.md` file by pasting in the contents of all the files in the repo and asking for a “comprehensive and user-friendly readme file that will guide a student through completing the assignment”. I also prompted Google Gemini 3 Pro to create a simple unit tests file given the functions in `src/filters.py`, since I’m not familiar with writing well-syntaxed python unit tests from scratch.

The rationale for using different models is that I’ve noticed Claude tends to be more verbose than necessary, while Gemini tends to be over-brief. Since I wanted a comprehensive readme and a succinct test file, the model choices made themselves.

I also learned this useful line from Gemini, which helped get all the “relative” imports working correctly with no hassle!

Python

```
# Add src to path so imports work
```

```
sys.path.append(os.path.abspath(os.path.join('..')))
```